Contribution Results

Dual-View Address Mapping for NPU–PIM Memory on Ramulator 2 — Group 13

Yash Gupta, Nikunj Ladumor, Vamshi Kumar Kanduri, Priyanshu Ganjeer — Dept. of CSE, IIT Guwahati

What is new over the baselines. (1) A PuLP/CBC MILP chooses the layout of the 15 line-index bits to maximize $\mathrm{NPU}(f) + \alpha\,\mathrm{PIM}(f)$ (we show α=0.25, NPU-leaning, and α=2, PIM-leaning). (2) `PFMRegionMapper`, a new Ramulator address mapper, gives every 2 MB region its own layout. NPU utilization is measured in Ramulator 2.1 (DDR4-2400R); PIM utilization is an analytical all-bank GEMV model.

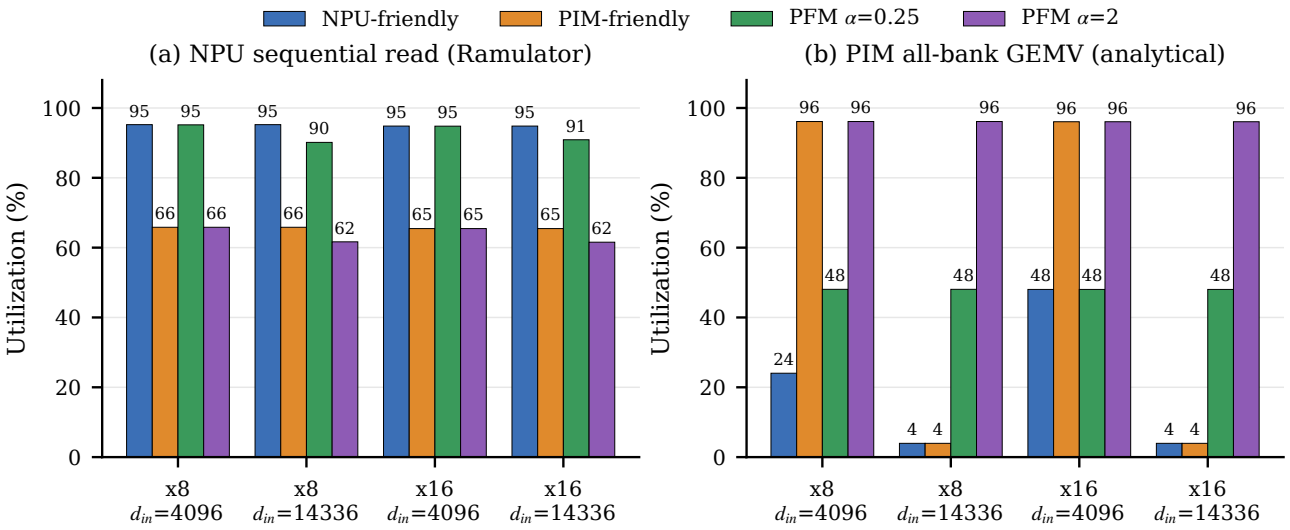


Figure 1: All layouts. The first two bars of each group are the baselines; green and purple are the optimized (PFM) layouts.

Table 1: Utilization (%) of every layout. The read-stream proxy (PIM order replayed as ordinary reads in Ramulator) is shown for reference.

Case	Layout	NPU	PIM	PIM proxy
x8, d_{in}=4096	NPU-friendly	95.23	24.03	91.25
x8, d_{in}=4096	PIM-friendly	65.83	96.12	95.03
x8, d_{in}=4096	PFM α=0.25	95.17	48.06	94.87
x8, d_{in}=4096	PFM α=2	65.83	96.12	95.03
x8, d_{in}=14336	NPU-friendly	95.23	3.95	51.66
x8, d_{in}=14336	PIM-friendly	65.83	3.95	27.85
x8, d_{in}=14336	PFM α=0.25	90.15	48.06	94.91
x8, d_{in}=14336	PFM α=2	61.66	96.12	94.98
x16, d_{in}=4096	NPU-friendly	94.82	48.03	94.83
x16, d_{in}=4096	PIM-friendly	65.46	96.06	94.86
x16, d_{in}=4096	PFM α=0.25	94.79	48.03	94.83
x16, d_{in}=4096	PFM α=2	65.46	96.06	94.86
x16, d_{in}=14336	NPU-friendly	94.82	3.95	53.03
x16, d_{in}=14336	PIM-friendly	65.46	3.95	36.66
x16, d_{in}=14336	PFM α=0.25	90.90	48.03	94.83
x16, d_{in}=14336	PFM α=2	61.56	96.06	94.85

Table 2: Comparison over the baseline: change in utilization (percentage points) of each optimized layout against the baseline it is closest to.

Case	Comparison	ΔNPU	ΔPIM
x8, d_{in}=4096	PFM α=0.25 vs NPU-friendly	-0.06	+24.03
x8, d_{in}=4096	PFM α=2 vs PIM-friendly	+0.00	+0.00
x8, d_{in}=14336	PFM α=0.25 vs NPU-friendly	-5.08	+44.11
x8, d_{in}=14336	PFM α=2 vs PIM-friendly	-4.17	+92.17
x16, d_{in}=4096	PFM α=0.25 vs NPU-friendly	-0.03	+0.00
x16, d_{in}=4096	PFM α=2 vs PIM-friendly	+0.00	+0.00
x16, d_{in}=14336	PFM α=0.25 vs NPU-friendly	-3.92	+44.08
x16, d_{in}=14336	PFM α=2 vs PIM-friendly	-3.90	+92.11
x8, whole layer	PFM α=0.25 vs NPU-friendly	-1.41	+29.44
x8, whole layer	PFM α=2 vs PIM-friendly	-1.12	+24.82
x16, whole layer	PFM α=0.25 vs NPU-friendly	-1.07	+11.87
x16, whole layer	PFM α=2 vs PIM-friendly	-1.05	+24.80

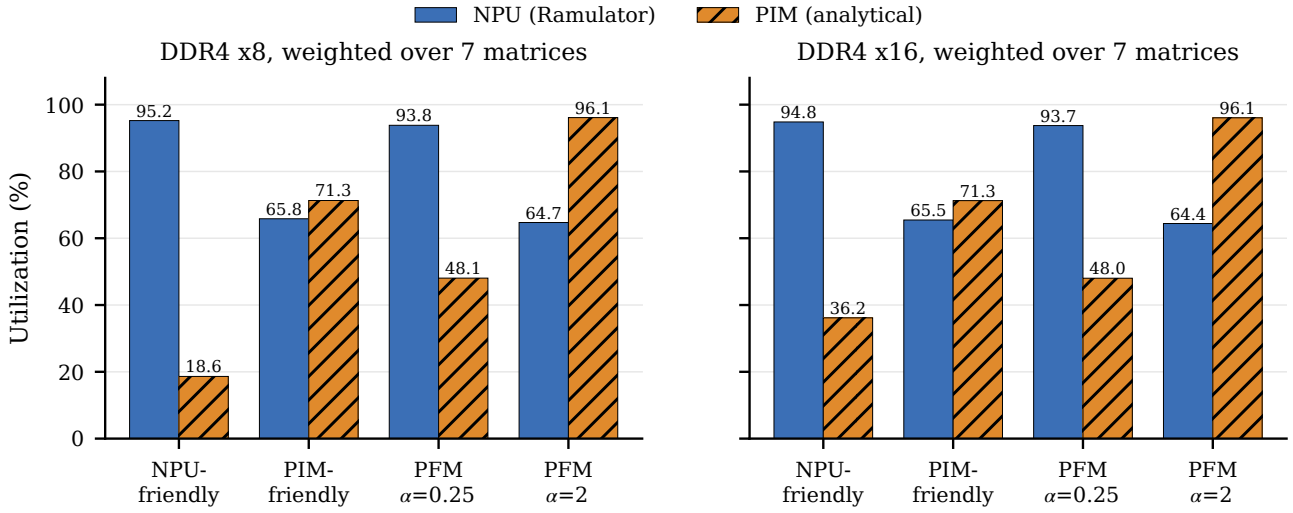


Figure 2: Whole LLaMA3-8B decoder layer: all seven weight matrices, weighted by their size in 2 MB pages of weight data.

Table 3: Whole decoder layer, utilization (%).

DRAM	Layout	NPU	PIM
x8	NPU-friendly	95.23	18.62
x8	PIM-friendly	65.83	71.30
x8	PFM α=0.25	93.82	48.06
x8	PFM α=2	64.71	96.12
x16	NPU-friendly	94.82	36.16
x16	PIM-friendly	65.46	71.26
x16	PFM α=0.25	93.74	48.03
x16	PFM α=2	64.41	96.06

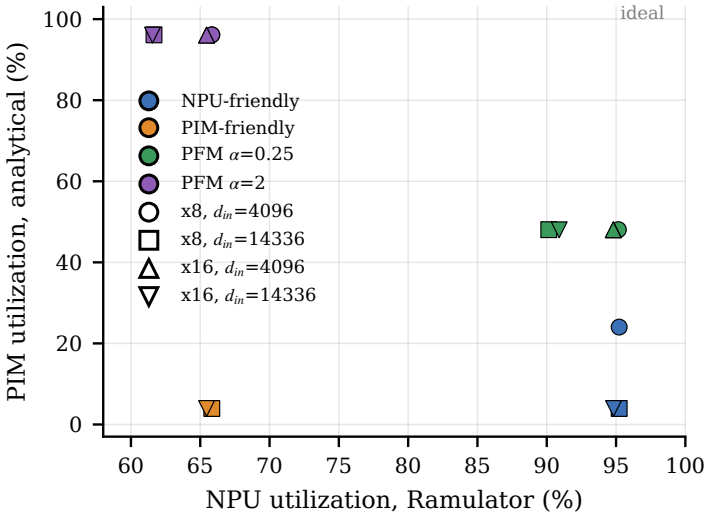


Figure 3: NPU vs. PIM utilization of every layout.

Table 4: `PFMRegionMapper` inside Ramulator: one NPU read stream over eight 2 MB regions (six d_{in}=4096 pages, two `down_proj` pages), DDR4 x8. For 20 of 20 layouts the mapper gives exactly the same cycle count as an independent Python implementation.

Plan	Layouts used	NPU %
NPU-friendly everywhere	1	95.05
PIM-friendly everywhere	1	65.81
PFM α=0.25 per region	2	93.70
PFM α=2 per region	2	64.70

Analysis and comparison over the baseline.

- **Whole layer (x8).** PFM α=0.25 keeps 93.8% NPU utilization against 95.2% for NPU-friendly (-1.4 points) while raising PIM utilization from 18.6% to 48.1% (+29.4 points). PFM α=2 raises PIM from 71.3% (PIM-friendly) to 96.1% (+24.8 points) for -1.1 NPU points.
- d_{in}=4096 **(six matrices).** PFM α=0.25 keeps a bank-group bit at bit 0 for the NPU and places the other three bank-level bits in the PIM unit-selection bits: NPU stays at 95.2% while PIM doubles from 24.0% to 48.1%. For $\alpha \geq 1$ the optimizer selects a layout equivalent to PIM-friendly.
- **`down_proj`.** Both baselines give 3.95% PIM. The optimizer moves the row bits out of the unit-selection bits: 48.1% PIM at 90.2% NPU (α=0.25) or 96.1% PIM (α=2). Part of this gain comes from adapting the layout to the column length, which the fixed baselines (designed for d_{in}=4096) do not do; the optimizer finds the shape-specific layout automatically.
- **x16** (two bank groups) shows the same pattern; on d_{in}=4096 the NPU-friendly layout already reaches 48% PIM there, so α=0.25 matches it rather than improving it.
- **Trade-off.** On single-channel DDR4 no layout is best for both devices: x8 has only four bank-level bits, the NPU needs one at bit 0 and PIM needs all four in its unit-selection bits, which caps PIM at 50% while the NPU runs at full speed. α chooses the point on this trade-off (switch at α=2/3 in our model).
- **Per-region mapping.** With a different layout in different regions of the same run, the α=0.25 plan reaches 93.7% NPU, matching the single-region results (93.9% expected), so per-region layouts cost almost nothing.
- **Caveats.** PIM utilization is analytical, not cycle-simulated; the memory is single-channel DDR4 rather than the paper's HBM2e-PIM, so the values do not reproduce the paper's Table III, only its trends.